

Visualisierung kinematischer Kennwerte

Philipp Mikos

April 2025

1 Einleitung

Die Grundlagen der Kinematik stellen einen wesentlichen Bestandteil der Robotik dar und sind sowohl für mobile als auch für stationäre Roboter von zentraler Bedeutung. Während es bei stationären Robotern häufig darum geht, die Position des Endeffektors präzise zu berechnen, konzentriert sich die Kinematik bei mobilen Robotern vor allem darauf, deren Position zuverlässig bestimmen zu können. Ein fundiertes Verständnis der erforderlichen kinematischen Modelle ist unerlässlich, um diese Größen korrekt zu berechnen. Siegwart et al., 2011, S. 57

Im Rahmen dieser Projektarbeit wird ein Framework entwickelt, das die Visualisierung kinematischer Kennwerte ermöglicht. Das Ziel dieses Frameworks ist es, unterstützend in der Lehre eingesetzt zu werden, um das Verständnis der Kinematik zu fördern und die praktische Anwendung dieser Konzepte zu erleichtern. Ein besonderer Fokus dieser Arbeit liegt auf der Rotation von Körpern im Raum. Dabei wird insbesondere erläutert, was unter Eulerwinkeln sowie Roll-Pitch-Yaw-Winkeln zu verstehen ist und worin die Unterschiede zwischen beiden Darstellungen bestehen. Darüber hinaus werden auch Rotationsmatrizen sowie Quaternionen behandelt, um die verschiedenen Ansätze zur Beschreibung von Rotationen umfassend darzustellen.

2 Motivation

Obwohl kinematische Modelle in vielen Bereichen der Robotik eine zentrale Rolle spielen, fällt es Studierenden häufig schwer, diese abstrakten mathematischen Konzepte vollständig zu durchdringen. Besonders Rotationen im dreidimensionalen Raum – etwa durch Eulerwinkel, Roll-Pitch-Yaw-Darstellungen, Rotationsmatrizen oder Quaternionen beschrieben – wirken in der Theorie oft wenig

anschaulich und sind ohne visuelle Unterstützung schwer intuitiv erfassbar Gutiérrez et al., 2016, S. 94–95.

In der Hochschullehre entsteht dadurch eine Lücke zwischen theoretischem Wissen und praktischem Verständnis. Lehrende stehen vor der Herausforderung, komplexe Zusammenhänge nicht nur formal korrekt, sondern auch nachvollziehbar und greifbar zu vermitteln. Gleichzeitig benötigen Studierende Möglichkeiten, das Gelernte interaktiv zu erproben, um eine tiefere, anwendungsorientierte Intuition für die Kinematik zu entwickeln.

Moderne Lehrmethoden, die auf Visualisierung und Interaktion setzen, können hier entscheidend zur Verbesserung des Lernerfolgs beitragen. Insbesondere browserbasierte und plattformunabhängige Lösungen, wie sie in JupyterLab realisierbar sind, eröffnen neue Perspektiven für eine zeitgemäße und praxisnahe Vermittlung technischer Inhalte Gutiérrez et al., 2016, S. 239–240, 407.

3 Ziel

Vor diesem Hintergrund zielt das Projekt darauf ab, ein interaktives Visualisierungs-Framework zu entwickeln, das die Lehre im Bereich der Robotik an der Fachhochschule Dortmund unterstützt und bereichert. Das Framework wird speziell für den Einsatz innerhalb der JupyterLab-Umgebung konzipiert und soll dazu dienen, kinematische Zusammenhänge auf intuitive und anschauliche Weise darzustellen.

Ein zentrales Anliegen ist es, Lehrpersonen und Aufgabenstellern ein Werkzeug an die Hand zu geben, mit dem sie durch einfache Python-Befehle individualisierbare Aufgabenstellungen und Übungsszenarien gestalten können. Diese sollen es Studierenden ermöglichen, eigenständig mit Parametern zu experimentieren, Bewegungsabläufe zu analysieren und dadurch ein tieferes Verständnis für die Wirkungsweise kinematischer Modelle zu entwickeln.

Besonderes Augenmerk liegt auf der Darstellung verschiedener Rotationsmodelle im Raum, darunter Eulerwinkel, Roll-Pitch-Yaw-Winkel, Rotationsmatrizen und Quaternionen. Durch deren visuelle Umsetzung werden nicht nur Unterschiede und Gemeinsamkeiten der jeweiligen Darstellungsformen sichtbar, sondern auch deren konkrete Bedeutung im räumlichen Kontext erfahrbar gemacht.

Insgesamt verfolgt das Projekt das Ziel, den Lernprozess im Bereich der Kinematik durch moderne, interaktive Lehrmittel zu unterstützen und einen Beitrag zu einer zeitgemäßen, verständnisorientierten Ausbildung in der Robotik zu leisten.

4 Theoretischer Hintergrund

4.1 Darstellung der Position und Orientierung eines Roboters

Ein Roboter wird in der Regel als ein unverformbarer, fester Körper (Rigid Body) betrachtet, der sowohl eine bestimmte Position als auch eine Orientierung im Raum besitzt. Die Position beschreibt den Abstand und die Verschiebung zwischen dem globalen Weltkoordinatensystem und dem körpereigenen Koordinatensystem des Roboters. Die Orientierung hingegen gibt den Unterschied in der Winkelausrichtung zwischen dem Weltkoordinatensystem und dem lokalen Koordinatensystem des Roboters an. Die Kombination aus Position und Orientierung eines Körpers wird als Pose bezeichnet. Die Transformation von einer Pose in eine andere kann intuitiv als Bewegung interpretiert werden. Diese Transformation setzt sich aus einer Translation (Verschiebung) und einer Rotation (Drehung) zusammen (Corke, 2023, S. 24–27). Speziell bei radgetriebenen Robotern wird die Pose auf einer Ebene beschrieben. In diesem Fall besteht die Pose aus den X- und Y-Koordinaten im globalen Koordinatensystem sowie dem Winkel zwischen dem globalen und dem körpereigenen Koordinatensystem. Damit lässt sich die Pose als ein Vektor aus diesen drei Elementen darstellen (Siegwart et al., 2011, S. 59).

4.2 Euler-Winkel und Roll-Pitch-Yaw-Winkel

Euler's Rotationstheorem besagt, dass zwei beliebige unabhängige, orthonormale Koordinatensysteme durch eine Abfolge von maximal drei Rotationen um Koordinatenachsen miteinander in Beziehung gesetzt werden können, wobei keine zwei aufeinanderfolgenden Rotationen um die gleiche Achse erfolgen dürfen (Kuipers, 1999, S. 136). Das bedeutet, dass jede Rotation zwischen zwei Koordinatensystemen im dreidimensionalen Raum durch eine Sequenz von drei Rotationswinkeln beschrieben werden kann, wobei jeder Winkel einer bestimmten Achse zugeordnet ist. Die Rotationen werden nacheinander ausgeführt: Zunächst erfolgt eine Rotation des Weltkoordinatensystems um eine Achse, wodurch ein neues Koordinatensystem entsteht. Anschließend wird dieses neue System um eine seiner eigenen Achsen rotiert, gefolgt von einer dritten Rotation um eine Achse des wiederum neu entstandenen Systems.

Insgesamt gibt es zwölf eindeutige Rotationsabfolgen. Sechs davon beinhalten Wiederholungen derselben Achse (aber nicht unmittelbar hintereinander), beispielsweise XYX , XZX , YXY , YZY , ZXZ oder ZYZ . Die anderen sechs Rotationsabfolgen verwenden jeweils alle drei Achsen einmal, beispielsweise XYZ , XZY , YZX , YXZ , ZXY oder ZYX .

Der Begriff Euler-Winkel wird häufig verwendet, um jede Art von dreifacher Rotationsdarstellung

zu bezeichnen. Diese Bezeichnung ist jedoch nicht präzise, da – abhängig von der Reihenfolge der Rotationen – verschiedene Darstellungen entstehen. Deshalb muss die genaue Rotationsabfolge angegeben werden, auch wenn innerhalb bestimmter technischer Fachgebiete meist eine stillschweigende Konvention herrscht.

Eine besondere Rolle spielen die sogenannten Roll-Pitch-Yaw-Winkel, bei denen es zwei gebräuchliche Rotationssequenzen gibt: ZYX und XYZ. In der mobilen Robotik wird typischerweise die ZYX-Sequenz verwendet, während bei stationären Roboterarmen oft die XYZ-Sequenz Anwendung findet Corke, 2023, S. 48–51.

4.3 Rotationsmatrizen

Eine Rotationsmatrix ist eine spezielle Art von Matrix, die ein Koordinatensystem um einen Winkel θ dreht und es dadurch in ein neues Koordinatensystem transformiert. Die Spalten einer Rotationsmatrix können dabei als die transformierten Einheitsvektoren des ursprünglichen Koordinatensystems interpretiert werden.

Rotationsmatrizen sind stets orthonormal, das bedeutet, ihre Spalten (bzw. Zeilen) stehen senkrecht aufeinander und haben die Länge Eins. Daraus folgt die wichtige Eigenschaft:

$$R^{-1} = R^T$$

wobei R^{-1} die Inverse und R^T die Transponierte der Matrix R ist. Ein weiteres charakteristisches Merkmal von Rotationsmatrizen ist, dass ihre Determinante stets den Wert 1 hat.

Rotationsmatrizen sind außerdem Elemente der sogenannten speziellen orthogonalen Gruppe der Dimension 2, die als $SO(2)$ bezeichnet wird. dies schreiben wir auch als

$$R \in SO(2) \subset \mathbb{R}^{2 \times 2}$$

Die spezielle orthogonale Gruppe ist eine Gruppe bezüglich der Matrixmultiplikation. Das bedeutet, dass das Produkt zweier Rotationsmatrizen wieder eine Rotationsmatrix ist und somit ebenfalls zur Gruppe gehört. Ebenso ist die Inverse jeder Rotationsmatrix ebenfalls Teil der Gruppe Corke, 2023, S. 33.

4.4 Quaternionen und Unit-Quaternionen

Quaternionen wurden 1843 von Sir William Rowan Hamilton entdeckt. Sie erweitern die komplexen Zahlen und bestehen aus einem Skalar- und einem Vektoranteil:

$$q = s + v_x i + v_y j + v_z k$$

wobei i , j und k besondere Einheiten mit $i^2 = j^2 = k^2 = ijk = -1$ sind. Quaternionen bilden einen Vektorraum: Man kann sie addieren, skalieren, multiplizieren (Hamilton-Produkt) und konjugieren. Die Multiplikation ist nicht kommutativ. Der Betrag eines Quaternions ist:

$$||q|| = \sqrt{s^2 + v_x^2 + v_y^2 + v_z^2}$$

Unit-Quaternionen sind Quaternionen mit Betrag 1. Sie werden häufig verwendet, um Rotationen im Raum darzustellen. Eine Rotation um eine Achse $\hat{v}$ mit Winkel θ wird als

$$q = \cos(\theta/2) + \hat{v} \sin(\theta/2)$$

bezeichnet.

Quaternionen bieten bei der Darstellung von Rotationen mehrere Vorteile. Sie ermöglichen eine recheneffiziente Kombination von Rotationen und vermeiden dabei Probleme wie den sogenannten Gimbal Lock, der bei der Verwendung von Euler-Winkeln auftreten kann. Zudem sind Quaternionen kompakter als Rotationsmatrizen und benötigen weniger Speicherplatz und Rechenleistung, was sie besonders attraktiv für Anwendungen in Bereichen wie Robotik, Computer Vision, Computergrafik und Navigation macht Corke, 2023, S. 59–61.

4.5 Differentialantrieb

Ein mobiler Roboter mit Differentialantrieb besitzt zwei angetriebene Räder, jeweils eines auf jeder Seite des Roboters. Größere Roboter können zusätzlich über ein oder mehrere weitere Räder auf jeder Seite verfügen, die entweder passiv sind oder mechanisch mit den aktiven Rädern gekoppelt werden, sodass sie sich einen Motor teilen. Die Räder sind starr am Roboter befestigt.

Die Steuerung des Roboters erfolgt durch die unterschiedliche Antriebsgeschwindigkeit der Räder. Drehen sich beide angetriebenen Räder mit gleicher Geschwindigkeit, bewegt sich der Roboter geradeaus. Rotieren die Räder hingegen mit unterschiedlichen Geschwindigkeiten, führt dies zu einer Drehbewegung des Roboters.

Bei Robotern mit Differentialantrieb wird üblicherweise davon ausgegangen, dass es sich bei den verbauten Rädern um Standardräder handelt. Das bedeutet, dass sich die Räder – und somit der gesamte Roboter – ausschließlich in x-Richtung bewegen können, während Bewegungen oder ein Rutschen in y-Richtung nicht möglich sind. Zudem wird angenommen, dass jedes Rad nur einen einzigen Kontaktpunkt mit der Ebene besitzt Corke, 2023, S. 140–141 Hertzberg et al., 2012, S. 107.

4.6 JupyterLab

JupyterLab ist eine leistungsstarke und erweiterbare Umgebung zur Erstellung und Bearbeitung von sogenannten computational notebooks. Es ist Teil des größeren Project Jupyter, das sich zum Ziel gesetzt hat, Werkzeuge und Standards für das interaktive Arbeiten mit Rechen-Notizbüchern zu entwickeln.

Ein computational notebook ist ein Dokument, das Computer-Code, Freitext-Beschreibungen, Daten, grafische Darstellungen (z.B. Diagramme, 3D-Modelle) sowie interaktive Bedienelemente miteinander kombiniert. In Verbindung mit einem Editor wie JupyterLab ermöglichen diese Notebooks eine schnelle, interaktive Entwicklung, die sich besonders gut für das Prototyping von Code, die Datenexploration, die Visualisierung und das Teilen von Ideen eignet.

JupyterLab ist eng verwandt mit anderen Anwendungen unter dem Dach von Project Jupyter, wie zum Beispiel Jupyter Notebook und Jupyter Desktop. Im Vergleich zu Jupyter Notebook bietet JupyterLab jedoch eine fortschrittlichere, funktionsreichere und individuell anpassbare Benutzererfahrung Jupyter, 2024.

5 Anforderungen

Das zu entwickelnde Framework soll in der Programmiersprache Python implementiert und für die Nutzung innerhalb der JupyterLab-Umgebung konzipiert sein. Es wird vorausgesetzt, dass Transformationen von Körpern im dreidimensionalen Raum visualisiert werden können. Diese Visualisierungen sollen bevorzugt direkt innerhalb von JupyterLab erfolgen. Darüber hinaus soll das Framework so gestaltet sein, dass es die komfortable Erstellung und Anwendung von Übungsaufgaben unterstützt und vereinfacht. Zusätzlich wird gefordert, dass das Framework mit SymPy kompatibel ist, um symbolische Berechnungen und deren Integration in die Visualisierungen zu ermöglichen.

Das Framework soll grundlegende Konzepte der räumlichen Transformation abdecken. Dazu zählen

insbesondere Rotationen, Eulerwinkel, Roll-Pitch-Yaw-Winkel sowie Quaternionen zur Darstellung und Analyse von Orientierungen im dreidimensionalen Raum. Darüber hinaus ist vorgesehen, auch einfache Modelle radgetriebener Roboter zu integrieren, um kinematische Zusammenhänge und Bewegungsabläufe anschaulich darzustellen.

Langfristig ist geplant, das Framework um Funktionalitäten zur Modellierung stationärer Roboter, insbesondere Knickarmroboter, zu erweitern. In diesem Zusammenhang sollen auch die Vorwärts- und Rückwärtstransformationen behandelt werden. Aufgrund des Umfangs und der Komplexität dieser Themen wird dieser Teil voraussichtlich im Rahmen einer späteren Bachelorarbeit realisiert und vertieft.

Zwar existieren bereits etablierte Werkzeuge wie beispielsweise Peter Corke's Robotics Toolbox for Python, welche eine Vielzahl an Funktionen für die Modellierung und Analyse robotischer Systeme bieten. Allerdings ist die Nutzung dieser Toolbox im Kontext von JupyterLab nur eingeschränkt möglich. Die dort verwendeten Visualisierungen basieren auf einem separaten Renderer namens Swift, welcher ein eigenes Browserfenster zur Darstellung benötigt und somit nicht direkt in JupyterLab eingebettet ist Corke, 2023, S. 77. Darüber hinaus ist die Toolbox sehr umfangreich und adressiert viele Konzepte, die für die Anforderungen und Inhalte dieses Projekts nicht von unmittelbarer Relevanz sind.

Eine maßgeschneiderte Lösung, die gezielt auf die Bedürfnisse und Themen des Lehrkonzepts an der Fachhochschule Dortmund zugeschnitten ist, bietet daher klare Vorteile. Sie erleichtert die Erstellung spezifischer Übungsaufgaben durch den Aufgabensteller, fördert die Anschaulichkeit durch eingebettete Visualisierungen für Lernende und trägt insgesamt zur Effizienz und Qualität der Lehre im Bereich der robotischen Transformationen bei.

6 Aufbau

6.1 Technischer Aufbau

Das Framework ist für die Nutzung innerhalb der JupyterLab-Umgebung ausgelegt. Die Fachhochschule Dortmund stellt hierfür einen eigenen JupyterLab-Server bereit, auf den sowohl über eine VPN-Verbindung als auch direkt im Hochschulnetz zugegriffen werden kann. Dadurch ist keine lokale Installation erforderlich, und die Arbeitsumgebung steht einheitlich auf verschiedenen Endgeräten zur Verfügung.

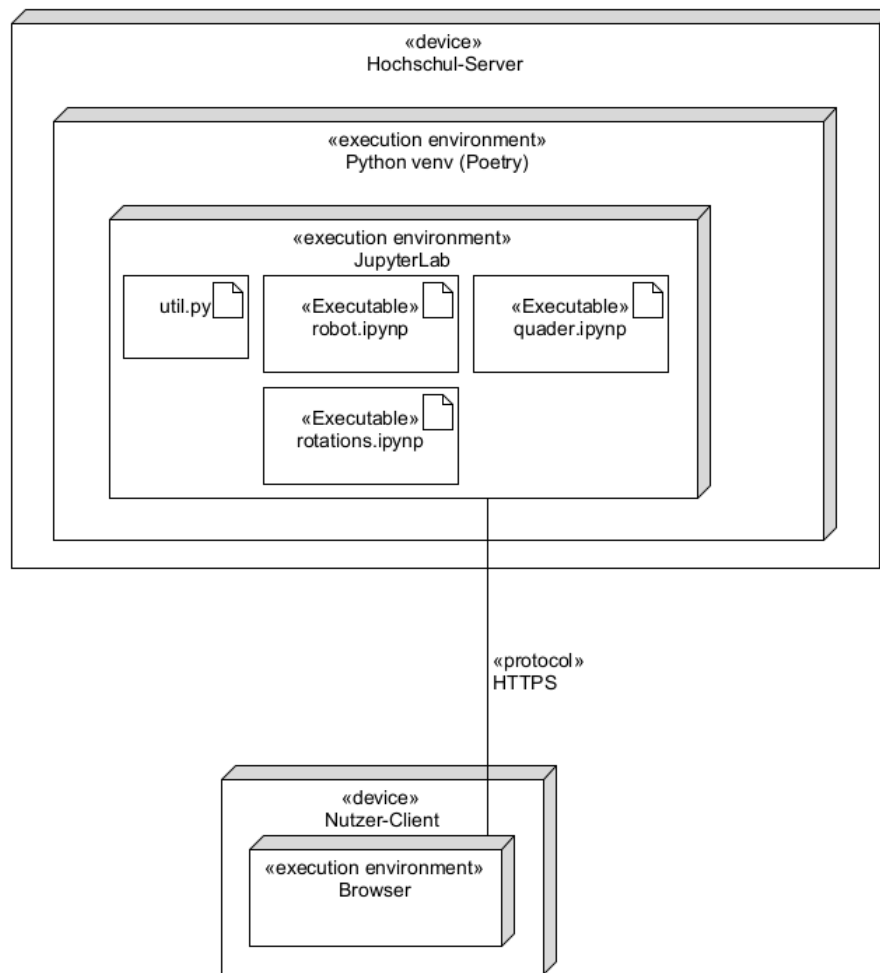
Um eine einfache Einrichtung des Frameworks auf dem Server zu gewährleisten, ist es erforderlich, Versionskonflikte zwischen den verwendeten Abhängigkeiten zu vermeiden. Zu diesem Zweck wird das Werkzeug Poetry eingesetzt. Poetry ermöglicht die Verwaltung aller benötigten Python-Bibliotheken in festen Versionen und erstellt eine isolierte virtuelle Umgebung, in der alle Abhängigkeiten konsistent installiert sind. Dadurch wird sichergestellt, dass das Framework unabhängig von der Zielumgebung stabil und reproduzierbar ausgeführt werden kann.

Eine der zentralen Bibliotheken innerhalb des Projekts ist NumPy, eine weit verbreitete Python-Bibliothek für numerische Berechnungen. NumPy wird im Kontext dieses Projekts hauptsächlich für erweiterte mathematische Operationen im Bereich der linearen Algebra eingesetzt.

Darüber hinaus ist pythreejs als weitere wichtige Abhängigkeit integriert. Es handelt sich hierbei um eine Python-Wrapper-Bibliothek für die JavaScript-Bibliothek three.js, die auf WebGL basiert. pythreejs ermöglicht die Darstellung und Interaktion mit 3D-Geometrien innerhalb eines WebGL-Renderers. Hierbei wird mithilfe von pythreejs JavaScript-Code generiert und im Hintergrund automatisch auf dem Client des Nutzers ausgeführt. Die Visualisierung erfolgt somit lokal im Webbrowser innerhalb des Notebooks, wo auch die Steuerung über Python stattfindet. Der Renderer kann daher nahtlos in der JupyterLab-Umgebung eingebettet und somit direkt in einem Jupyter Notebook visualisiert und gesteuert werden. Außerdem können auf diese Weise Geometrien mit Grafikbeschleunigung gerendert werden.

Durch den Einsatz dieser Technologien wird eine effiziente und plattformunabhängige Darstellung sowie Manipulation komplexer geometrischer Strukturen innerhalb des Frameworks ermöglicht.

Das Zusammenspiel der verschiedenen Komponenten kann der Abbildung 1 entnommen werden.



Kategorie	Technologie	Beschreibung
Laufzeitumgebung	JupyterLab	Web-basierte Entwicklungsumgebung für interaktive Notebooks
Paketmanager	Poetry	Verwaltung von Abhängigkeiten und virtuellen Umgebungen
Mathe-Bibliothek	Numpy	Bibliothek für numerische Berechnungen und lineare Algebra
3D-Bibliothek	pythreejs	Python-Wrapper für three.js zur Darstellung von 3D-Geometrien in Notebooks
Frameworkinterne-Bibliothek	util.py	Eigene Datei mit Kernfunktionen zur Geometrie-Erzeugung und -Animation

Tabelle 1: Eine einfache Tabelle

6.2 Framework-Library

Die Bibliothek des Frameworks besteht aus einer zentralen Datei, `util.py`. In dieser Datei sind sämtliche Kernfunktionen implementiert, die zur Erstellung, Transformation und Animation geometrischer Formen benötigt werden. Darüber hinaus enthält `util.py` eine Reihe von Hilfsfunktionen, die unter anderem die Umrechnung zwischen verschiedenen Darstellungsformen von Rotationen ermöglichen sowie die Erstellung von Koordinatensystemen unterstützen.

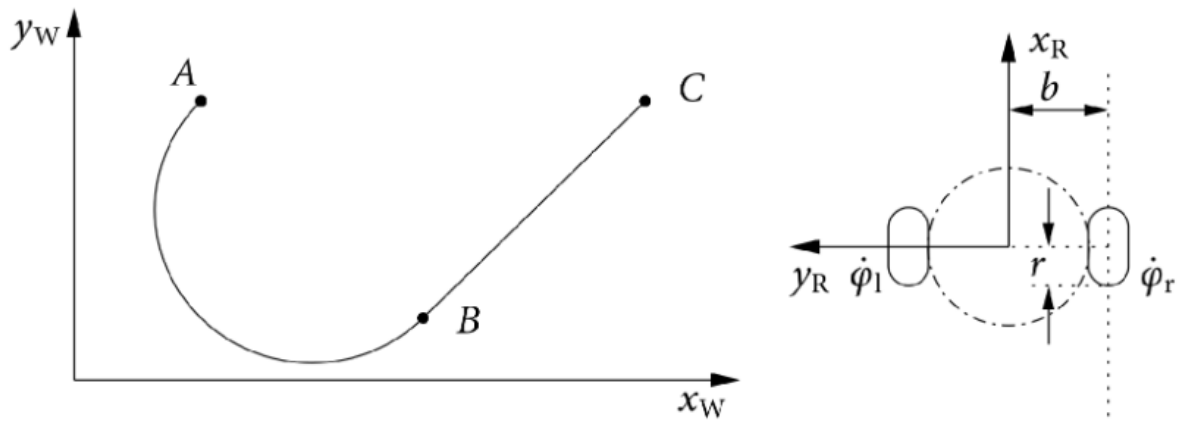
6.3 Beispiel-Notebooks

Das Framework enthält drei Beispiel-Notebooks, die die Anwendungsmöglichkeiten und Funktionalität praxisnah demonstrieren. Sie sollen dem Aufgabensteller als Orientierung dienen um mithilfe des Frameworks Aufgaben oder Umgebungen zum Lernen zu generieren.

6.3.1 `robot.ipynb`

Eines dieser Notebooks trägt den Namen `robot.ipynb` und dient der Veranschaulichung einer einfachen Steuerungsaufgabe. In diesem Notebook wird ein dreidimensional visualisierter Roboter mit Differentialantrieb dargestellt. Zusätzlich wird ein vorgegebener Pfad angezeigt, den der Roboter abfahren soll. Die Aufgabe besteht darin, die entsprechenden Antriebsgeschwindigkeiten zu berechnen und anschließend die Funktion `move(robot, vel, steps)` mit diesen Werten aufzurufen. Das Notebook Orientiert sich an einer Übungsaufgabe der Fachhochschule Dortmund im Modul Robotik aus dem Jahr 2024

Gegeben ist ein mobiler Roboter mit Differentialantrieb, der den im Weltkoordinatensystem abgebildeten Kurs von A nach C vorwärts abfährt. Der Kurs besteht aus einem Halbkreis zwischen den Punkten $A = (4, 9)$ und $B = (11, 2)$ und zwischen B und $C = (18, 9)$ aus einer Geraden. Die Positionen der Punkte sind in m angegeben.

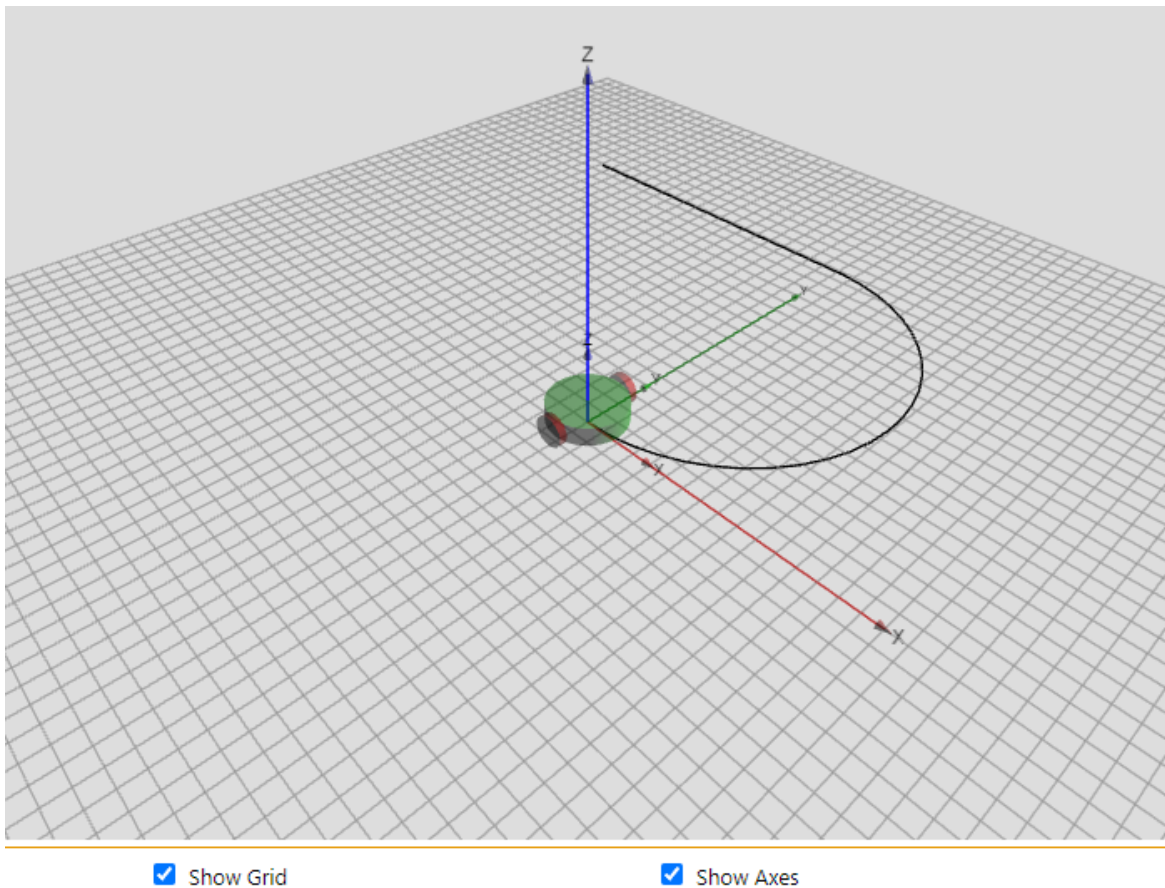


Der Roboter soll den gesamten Kurs mit einer konstanten Lineargeschwindigkeit von $\dot{x}_R = 1 \frac{m}{s}$ abfahren. Beschleunigungs- und Verzögerungsvorgänge sollen vernachlässigt werden. Der Roboter hat einen halben Radabstand von $b = 0.5m$ und Radradius von $0,25m$.

[...]

Welche Geschwindigkeiten in Roboterkoordinaten ($v_R = \dot{x}_R = (\dot{x}_R, \dot{y}_R, \dot{\theta})^T$) ergeben sich für die beiden Abschnitte jeweils? [...]

Nach Ausführung der jeweiligen Zelle beginnt der Roboter mit der Bewegung. Die Nutzerin bzw. der Nutzer kann dabei direkt im Jupyter Notebook beobachten, ob der Roboter dem gewünschten Pfad korrekt folgt.

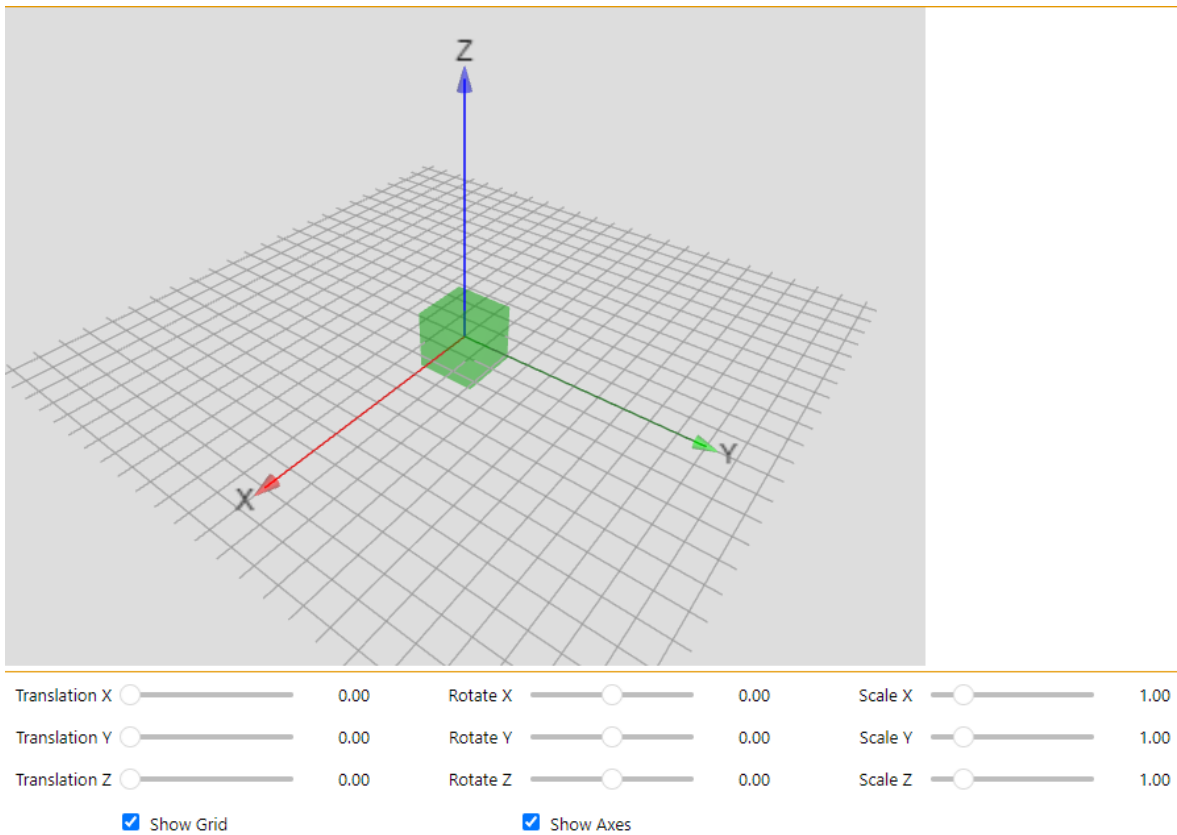


6.3.2 rotations.ipynb

Ein weiteres Beispiel-Notebook trägt den Namen `rotations.ipynb`. In diesem Notebook wird ein dreidimensionaler Würfel visualisiert, der vom Nutzer interaktiv transformiert werden kann.

Über integrierte Schieberegler lassen sich Translation, Rotation und Skalierung des Würfels in Echtzeit anpassen. Ziel dieses Notebooks ist es, die Möglichkeiten des Frameworks zur Darstellung und Simulation geometrischer Transformationen zu veranschaulichen.

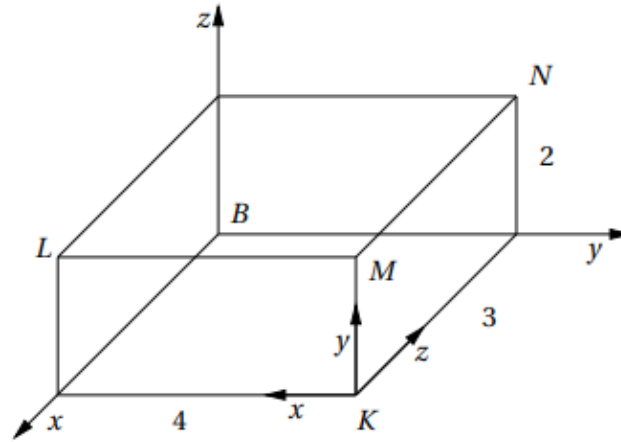
Die direkte visuelle Rückmeldung innerhalb des Jupyter Notebooks ermöglicht es dem Lernenden, die Auswirkungen der jeweiligen Transformationen unmittelbar nachzuvollziehen und ein besseres Verständnis für deren Wirkung im dreidimensionalen Raum zu entwickeln.



6.3.3 quader.ipynb

das andere Beispiel-Notebook trägt den Namen `quader.ipynb`. Es Orientiert sich an einer Klausuraufgabe der Fachhochschule Dortmund im Modul Robotik aus dem Jahr 2010, in der ein Quader um die Achsen des Basis-Koordinatensystems rotiert, die jeweiligen Rotationsmatrizen aufgestellt und zu einer Gesamtrrotationsmatrix zusammengefasst werden sollen.

Gegeben ist ein quaderförmiger Körper mit den Kantenlängen $l_x = 3$, $l_y = 4$ und $l_z = 2$. Eine Quaderecke befindet sich, wie in der nachfolgenden Abbildung dargestellt, im Ursprung eines ortsfesten Koordinatensystems B . In der Quaderecke K ist ein körperfestes Koordinatensystem K angebracht.

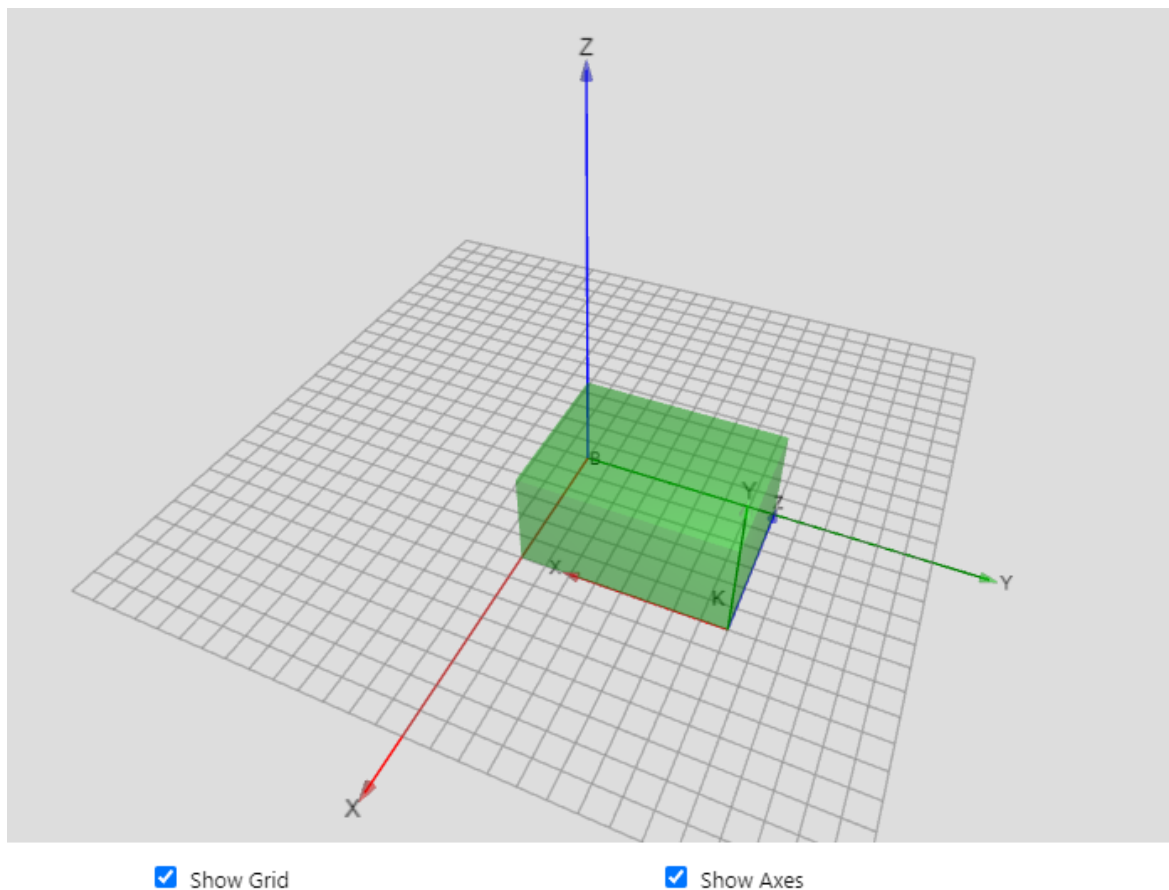


Der Körper wird nun zunächst um den Winkel $\phi_z = 60^\circ$ um die z -Achse, dann um den Winkel $\phi_x = -180^\circ$ um die x -Achse und schließlich um den Winkel $\phi_y = -90^\circ$ um die y -Achse des ortsfesten Koordinatensystems B gedreht.

- Stellen Sie die Rotationsmatrizen $R_x := R(x, -180^\circ)$, $R_y := R(y, -90^\circ)$ und $R_z := R(z, 60^\circ)$ auf.
- Berechnen Sie die Rotationsmatrix R_G für die Gesamttrotation.
- Welche Positionen nehmen nach der Drehung des Körpers die Körperecken M und N in Bezug auf das Koordinatensystem B ein?
- Welche Orientierung, ausgedrückt als Rotationsmatrix, hat das Koordinatensystem K vor der Drehung der Körpers bezüglich der Koordinatensystems B ?
- Berechnen Sie die Eulerschen Winkel der Rotationsmatrix vor der Drehung.
- Welche Orientierung, ausgedrückt als Rotationsmatrix, hat das Koordinatensystem K nach der Drehung des Körpers bezüglich des Koordinatensystems B ?
- Berechnen Sie die Roll-Nick-Gier-Winkel der Rotationsmatrix von K nach der Drehung.

Das Notebook dient der Visuellen Simulation der Rotationen. Mit der Funktion `rotate_global_animated(mesh, angles, order)` wird der Quader zunächst wie in der Aufgabe beschrieben um die Achsen des Koordinatensystems B Rotiert. Diese Transformation wird Visuell dargestellt um ein Gefühl für die Rotation zu vermitteln und die Endposition aufzuzeigen. Anschließend werden die Matrizen als Variablen aufgestellt, zusammengefasst und an die Funktion `apply_rot_matrix_animated(mesh, rot_mat, speed, show_rot_mat)` übergeben, sodass die Rotation des Quaders anhand der Rotationsmatrix betrachtet

und mit der vorrangegangenen Transformation verglichen werden kann. Dieses Notebook kann vom Aufgabensteller so modifiziert werden, dass die Matrizen von den Lernenden ausgefüllt werden müssen.



Literatur

- Corke, P. (2023). *Robotics, Vision and Control: Fundamental Algorithms in Python* (Third edition, Bd. 146). Springer International Publishing AG.
- Gutiérrez, Á., Leder, G. C., Boero, P., Boero, P., Gutiérrez, Á., & Leder, G. C. (2016). *The Second Handbook of Research on the Psychology of Mathematics Education: The Journey Continues* (1. Aufl.). Springer.
- Hertzberg, J., Lingemann, K., & Nüchter, A. (2012). *Mobile Roboter : eine Einführung aus Sicht der Informatik*. Springer.
- Jupyter, P. (2024). *JupyterLab Documentation*. Verfügbar 14. April 2025 unter <https://jupyterlab.readthedocs.io/en/stable/>

- Kuipers, J. B. (1999). *Quaternions and Rotation Sequences : A Primer with Applications to Orbits, Aerospace and Virtual Reality*. Princeton University Press.
- Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to autonomous mobile robots* (2. ed.). MIT Press.